

CM2015 Programming with Data: Mid-Term Project Report

Domain: Automated Starbucks Order Assistant

Student Name: Alishba Masood

Degree: BSc Computer Science

1. Project Use-Case & Operational Domain (Rubric #1 & #11)

The developed system is a data-driven conversational agent that simulates the ordering workflow of a coffee shop, modelled on a Starbucks-style counter interaction. In this domain, the chatbot's job is to move a customer through a fixed sequence of ordering decisions — name capture, drink selection, size, a bakery side, and payment — while remaining flexible enough to handle a change of mind at any point in that sequence.

The agent handles greeting the customer, capturing their name, parsing drink and quantity selections, confirming size and bakery add-ons, routing payment method, and handling requests to add to or amend an order already in progress. All patterns, response templates, and synonym mappings are kept out of the executable code and stored externally in a structured JSON configuration file (`intents.json`), so the conversational content can be edited without touching the underlying logic.

2. Code Architecture & Component Organisation (Rubric #3 & #4)

The implementation separates each stage of the conversational pipeline into a standalone function with a clear input/output contract, so the main loop simply orchestrates calls between them rather than containing any matching or extraction logic itself:

- **Data loading** (`load_files`): Reads `intents.json` and builds three dictionaries used everywhere else — `rgx2int` (regex pattern string → intent tag), `int2res` (intent tag → list of response templates), and `synonym_rules` (synonym word → canonical base word). It also derives a `greeting_words` set directly from the greeting intent's own patterns, rather than hard-coding it, so the data file stays the single source of truth.
- **Preprocessing** (`preprocess_input`): The normalisation gateway that every raw message passes through before any matching happens.
- **Entity extraction** (`extract_named_entities`): Reads the cleaned input and writes any recognised order details (name, item, size, side, quantity) into a persistent `user_memory` dictionary shared across turns.
- **Intent matching** (`match_intent`): Kept strictly separate from response generation — its only job is to decide which intent tag this turn belongs to.
- **Response generation** (`generate_response`) and the **main loop** (`chatbot`): `generate_response` chains the above functions together, then picks a response via `random.choice()` and fills in template placeholders; `chatbot()` runs the `while True` loop that calls it once per turn until the user types “exit” or “quit”.

3. Natural Language Preprocessing Pipeline (Rubric #8)

Every user message passes through `preprocess_input()` before matching takes place:

- **Normalisation**: the input is lowercased and apostrophes are stripped, so contractions like “I'm” and possessive/typo variants don't create separate, unmatched forms.

- **Synonym substitution:** synonym keys (e.g. "howdy", "macchiato", "medium") are sorted longest-first and compiled into a single regex, so a multi-word synonym is matched and replaced with its base form in one pass rather than several separate substitutions that could interfere with each other.
- **Punctuation removal, tokenisation, and lemmatisation:** remaining punctuation is stripped, the text is split into tokens with `nltk.word_tokenize()`, and each token is reduced to its root verb form with `WordNetLemmatizer(pos='v')` (e.g. "wanted" → "want"), so small variations in phrasing don't cause a pattern miss.

A `DISPLAY_NAMES` lookup table then restores correct brand casing when an item is later shown back to the user — this exists because Python's built-in `.title()` method cannot correctly capitalise "caffe latte" into "Caffè Latte" (it has no way to know the accented è), so the mapping is stored explicitly instead of derived at runtime.

4. Regular Expressions & Dynamic State Memory (Rubric #5, #6, & #7)

Pattern recognition is built entirely on regular expressions, using several distinct regex features:

- **Word-boundary enforcement:** every intent pattern is compiled as `\b(pattern)\b`, so a short pattern like "no" cannot accidentally match inside an unrelated word.
- **Alternation across an entire intent:** all patterns belonging to one intent (e.g. every greeting variant) are joined with `|` into a single compiled regex string.
- **Quantifiers and non-capturing groups for quantities:** `extract_named_entities()` uses `r"\b(\d+|one|two|three|four|five)\s+(?:caramel macchiato|caffe latte|pink drink|frappuccino|cold brew|latte|coffee|drink|item)s?\b"` to catch both numeric and written quantities (e.g. "2 macchiatos" or "two lattes").

Recognised details are written into the `user_memory` dictionary (name, item, size, side, quantity). Response templates in `intents.json` contain placeholders such as `{name}` and `{item}`, which are filled in at runtime with `selected_template.format(**user_memory)`. Because this dictionary persists for the whole session rather than being rebuilt each turn, a value captured early — the customer's name, for instance — is still available many turns later, including after the customer changes their mind about the order.

5. Iterative Development & Process Reflection (Rubric #10)

I built this project in four explicit stages, each one building on the last, following the scaffolded structure of the assignment itself:

- **Part 1 — Conversation loop:** I started with a minimal `while True` loop that read input with `input()` and checked it against a flat `precepts` list, using the matched index to look up a paired string in a separate `responses` list. I handled termination with a simple "bye" check. At this stage I had no regex, no word-boundary checking, and no memory of anything the user had said in earlier turns.
- **Part 2 — Pattern matching and modularisation:** I introduced the `re` module (word-boundary and alternation patterns such as `r"\b(hello|hi|hey|howdy|yo)\b"`), used `zip()` to pair the precept and response lists into a proper dictionary, and split my logic into two dedicated functions — `detect_pattern()` for matching and `chatbot_response()` for building the reply. I also introduced my first persistent memory here: a global `user_state` dictionary tracking name and color, populated using regex capture groups (`.group(1)`) and injected back into responses with `.format()`.
- **Part 3 — Externalising data into JSON:** I moved all patterns and responses out of my Python code and into `intents.json`, then built the `rgx2int/int2res` lookup structures from it. My first working loader

used pandas' `pd.read_json()` to read the file; I later replaced this in my Part 4 rebuild with Python's built-in `json.load()` instead, since it gives direct control over the file's structure without the overhead of a DataFrame for what is fundamentally a small, nested config file.

- **Part 4 — NLTK preprocessing and synonym handling:** I added tokenisation, lemmatisation, and single-pass synonym substitution, and rebuilt my final Starbucks ordering flow on top of this pipeline.

Feedback Received

I did not receive external feedback during development; my main source of iteration was self-directed testing, described below.

Bug discovery and resolution. During testing of Part 4, I found an edge case where the very first word I typed after the welcome message could silently corrupt the conversation state. My original `extract_named_entities()` logic captured *any* single word as the customer's name as a fallback, with no check on what that word actually was. Typing "hi" as a greeting was therefore stored as the name "Hi", and the bot's internal `is_awaiting_name` flag flipped to `False` one turn early. Every message after that was routed through my general intent matcher instead of the name-capture flow, so a real name like "alishba" matched nothing and fell through to the generic "I'm sorry" response, while a combined message like "hi alishba" simply re-matched the greeting pattern and looped forever.

I fixed this by restructuring both sides of the interaction: `match_intent()` now strips known greeting words out of the input before deciding whether a turn is a name response or a repeated greeting, and `extract_named_entities()` only ever captures a name when the turn was already classified as a name response — I removed the old "any lone word" fallback entirely. I then added handling for contractions (e.g. "I'm alishba") and a small filler-word list ("i", "am", "my", etc.) so these aren't mistaken for the name itself. I locked this behaviour in place with a dedicated regression test (Test 5 below).

I also ran a second, smaller round of testing focused on late-order changes: verifying that a customer who asks to change their drink, size, or side — even at the very last step, while being asked for a payment method — is routed back into the ordering flow correctly, without their name or any already-confirmed details being lost or reset. This is discussed further in Test Case 3 below.

6. System Verification & Test Cases (Rubric #11)

The notebook runs an automated diagnostic suite (`run_diagnostic_tests()`) with five hard assert statements before the interactive loop starts, covering greeting resolution, synonym-based item extraction, post-payment confirmation matching, and the two name-capture edge cases described above. Three representative cases are detailed below.

Test Case 1: Greeting Combined With a Name in One Turn

Input: While the bot is waiting for a name, the customer replies "hi alishba" in a single message, combining a greeting word with their actual name.

Execution: `match_intent()` strips the recognised greeting word ("hi") from the tokenised input and finds one token left over ("alishba"), so it classifies the turn as `name_response` rather than a repeated greeting. `extract_named_entities()` then captures the remaining word as the name.

Output: Stored Name: 'Alishba', and the bot proceeds to "Hi Alishba! What handcrafted beverage can I get started for you today?..." instead of repeating the greeting indefinitely, as it did before the fix.

Test Case 2: Synonym-Based Item and Quantity Extraction

Input: "i want 2 macchiato"

Execution: `preprocess_input()` maps the synonym "macchiato" to its base form "caramel macchiato" before lemmatisation. `extract_named_entities()` then matches both the quantity pattern ("2") and the item pattern against the normalised text, and looks the matched item up in `DISPLAY_NAMES` for correct display casing.

Output: `user_memory['item'] == 'Caramel Macchiato'`, confirming the synonym was resolved to the correct base item and given proper display formatting, independent of the quantity that was extracted alongside it.

Test Case 3: Mid-Order Change at the Payment Step

Input Context: The customer completes a full order — a venti Caramel Macchiato with a croissant — and is asked "Will you be paying via cash or card today?". Instead of answering, they reply: "wait, i want to change my order".

Execution: This message matches the `modify_order` intent rather than `payment_selection`, so the bot asks what they'd like to change instead of trying (and failing) to interpret it as a payment method. The customer then names a new drink ("cold brew"), a new size, and a new side in the following turns; each reply matches the same `item_selection/size_selection/side_selection` intents used during the original order, so no special-case branching was needed to support this — the existing pipeline simply re-enters the same flow.

Output: Throughout the whole exchange, `user_memory['name']` stays 'Alishba' and is never reset. The bot walks back through size and side confirmation for the new drink, then returns to "Will you be paying via cash or card today?", and the final receipt correctly reflects the updated order (grande Cold Brew), not the original one. A second scenario — asking to swap only the bakery side ("actually can i get a cookie instead") at the same payment step — was also verified: because it names a specific side item directly, it updates only that field and returns straight to the payment question, without re-asking for size unnecessarily.